

Studying Domain Randomization Strategies for Sim-to-Sim Transfer in Reinforcement Learning

Carlo Squarcia Mateo S350750
s350750@studenti.polito.it

Abstract—Reinforcement learning (RL) policies trained in simulation often experience performance degradation when deployed under modified dynamics. Domain Randomization (DR) is a common approach to mitigate this issue by exposing policies to variations of environment parameters during training. In this project extension, we study Uniform Domain Randomization (UDR) for sim-to-sim transfer in a custom MuJoCo Hopper environment. Beyond standard return-based evaluation, we introduce a stability-oriented analysis that examines failure dynamics, near-fall behavior, and internal latent representations. Our results show that moderate domain randomization improves robustness not only by increasing target performance, but also by inducing domain-invariant representations and larger stability margins.

I. INTRODUCTION

Reinforcement learning (RL) policies trained in simulation often suffer from performance drops when deployed under altered dynamics, even when changes are mild (e.g., mass/inertia shifts). A common approach to mitigate this gap is *Domain Randomization* (DR), where environment parameters are randomized during training to encourage robustness. This extension investigates the effectiveness and limitations of DR and Uniform Domain Randomization (UDR) for sim-to-sim transfer in a *custom Hopper* setting.

Contributions. We: (i) implement Uniform Domain Randomization (UDR) for physical parameter randomization in a custom Hopper environment; (ii) analyze the effect of randomizing different body components and randomization magnitudes on sim-to-sim transfer; (iii) study the interaction between policy network capacity and domain randomization; (iv) introduce a stability-oriented evaluation based on failure dynamics and near-fall behavior; (v) analyze internal latent representations using PCA and t-SNE to explain robustness effects beyond return-based metrics.

Most existing studies evaluate the effectiveness of domain randomization primarily through aggregate performance metrics such as average episodic return. However, these metrics provide limited insight into how robustness emerges and how policies fail under dynamics mismatch. In this extension, we explicitly focus on qualitative aspects of policy behavior, including long-horizon stability, failure dynamics, and internal representations learned by the policy network. This perspective allows us to better understand the mechanisms through which domain randomization affects sim-to-sim transfer.

II. RELATED WORK

Domain Randomization has been widely adopted as a practical approach to bridge the gap between training and deploy-

ment environments in reinforcement learning. By exposing policies to a range of dynamics variations during training, DR encourages the emergence of robust behaviors that generalize beyond a single nominal model.

This idea has been successfully applied in sim-to-real and sim-to-sim transfer settings, particularly for robotic locomotion tasks. Proximal Policy Optimization (PPO) is a commonly used algorithm in this context due to its stability and sample efficiency. More recent work has explored adaptive and robust variants of domain randomization, aiming to automatically tune parameter distributions or optimize for worst-case performance.

III. METHODOLOGY

A. Environment Setup: Source and Target Domains

We consider a custom MuJoCo Hopper environment with two distinct domains: a source domain used for training and a target domain used exclusively for evaluation. The source environment corresponds to the nominal Hopper model, where all physical parameters are fixed to their default values. The target environment introduces a dynamics shift by modifying the masses of selected body components, resulting in different inertial properties and contact dynamics.

The following body parts are considered for mass modification:

- Torso
- Thigh
- Leg
- Foot

In the target environment, the masses of these components are set to fixed values that differ from the nominal configuration, creating a sim-to-sim transfer scenario.

B. Domain Randomization Strategy

Domain Randomization (DR) is applied during training by sampling physical parameters at the beginning of each episode. Given a nominal mass value m_0 , the randomized mass m is sampled uniformly according to:

$$m \sim \mathcal{U}((1 - \delta)m_0, (1 + \delta)m_0), \quad (1)$$

where δ controls the strength of the randomization.

We investigate different DR configurations by varying both the subset of body parts being randomized and the randomization magnitude δ . Uniform Domain Randomization (UDR) is used throughout, meaning that all randomized parameters are sampled independently from uniform distributions.

C. Reinforcement Learning Algorithm

All policies are trained using Proximal Policy Optimization (PPO) with an actor-critic architecture. The policy and value functions are parameterized by multilayer perceptrons (MLPs) with ReLU activations.

To ensure fair comparison, all hyperparameters are kept fixed across experiments except for the variables under study (randomized body parts, randomization strength, and network architecture).

D. Stability-Oriented Analysis

In addition to standard return-based evaluation, we perform a stability-oriented analysis during policy evaluation. For each episode, we record the evolution of key physical quantities such as torso height and torso angle. From these trajectories, we compute metrics including time-to-failure, minimum torso height reached during the episode, and the number of near-fall events. Near-falls are defined as events in which the torso height drops below a predefined safety threshold without triggering episode termination. This analysis allows us to distinguish between abrupt failures and progressive instability, providing a more informative assessment of robustness.

IV. EXPERIMENTAL SETUP

A. Training Protocol

All experiments are conducted by training policies in the source environment. Domain Randomization, when enabled, is applied at the beginning of each episode via environment reset.

Training is performed independently for each experimental configuration. Unless otherwise specified, all PPO hyperparameters are identical across experiments.

B. Evaluation Protocol and Metrics

Policy performance is evaluated using the average episodic return. For each trained policy, evaluation is performed over multiple episodes in both the source and target environments. We report the mean return and standard deviation across episodes and random seeds.

To assess generalization and robustness, we consider the following evaluation settings:

- **Source** $\rightarrow$ **Source**: training and testing in the source environment.
- **Source** $\rightarrow$ **Target**: training in the source environment and testing in the target environment.

C. Failure and Stability Metrics

To analyze policy robustness beyond episodic return, we introduce several stability-related metrics. Specifically, we report the episode length and time-to-failure, defined as the number of steps until termination. We also record the minimum torso height reached during each episode and the standard deviation of the torso angle. Finally, we count near-fall events, defined as occurrences where the torso height falls below a safety margin but the episode does not immediately terminate. These metrics provide insight into how close a policy operates to instability and how errors accumulate over time.

D. Latent Representation Analysis Setup

In order to analyze how domain randomization affects the internal behavior of the policy network, we perform a latent representation analysis. For selected trained policies, we extract the activations of the penultimate layer of the policy network during evaluation rollouts. These activations represent the internal state encoding used by the policy to select actions.

Latent representations are collected by executing each policy in both the source and target environments over multiple evaluation episodes. The resulting high-dimensional latent vectors are then projected to two dimensions using Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE). PCA is used to analyze global structure and variance in the latent space, while t-SNE is employed to inspect local neighborhood relationships. This analysis allows us to assess whether domain randomization leads to domain-invariant internal representations.

E. Experiment 1: Impact of Randomized Body Parts

The first experiment investigates which body components are most important to randomize in order to improve sim-to-sim transfer performance. We train separate PPO policies by applying domain randomization to individual body parts independently, namely the torso, thigh, leg, and foot. Additionally, we evaluate a configuration where all body components are randomized simultaneously.

All experiments are conducted using the same training setup and randomization strength. Performance is evaluated in both source and target environments to assess the contribution of each body part to robustness. Based on the obtained results, randomizing the foot alone and randomizing all body components emerge as the most effective configurations, and are therefore selected for further analysis in subsequent experiments.

F. Experiment 2: Effect of Randomization Strength

In the second experiment, we analyze the influence of the randomization magnitude on policy robustness. Based on the best-performing configurations from the first experiment (foot-only randomization and full-body randomization), we train policies using different values of the randomization coefficient:

$$\delta \in \{0.05, 0.10, 0.20, 0.40\}.$$

This experiment aims to identify the trade-off between insufficient randomization, which may fail to cover the target dynamics, and excessive randomization, which may hinder learning stability and final performance.

G. Experiment 3: Effect of Policy Network Architecture

The third experiment studies the impact of policy network capacity on learning and generalization. We evaluate three different MLP architectures for the policy and value networks:

$$[64, 64], \quad [128, 128], \quad [256, 256].$$

For each architecture, training is repeated using three different random seeds. All other aspects of the training setup, including environment configuration and randomization strategy,

are kept fixed. This experiment allows us to analyze whether larger network capacity improves robustness under domain randomization or leads to diminishing returns.

V. RESULTS AND DISCUSSION

A. Experiment 1: Which Body Parts Should Be Randomized?

We first analyze the effect of randomizing individual body parts on sim-to-sim transfer performance. Table I reports the evaluation results when applying domain randomization to a single body component at a time, as well as when randomizing all components jointly. The baseline corresponds to training without domain randomization.

TABLE I
EFFECT OF RANDOMIZING INDIVIDUAL BODY PARTS ($\delta = 0.2$).

Randomized Parts	Source Return	Target Return
None (Baseline)	1569 $\pm$ 6	835 $\pm$ 18
Thigh only	1381 $\pm$ 184	1039 $\pm$ 29
Leg only	1016 $\pm$ 117	708 $\pm$ 15
Foot only	1523 $\pm$ 6	1266 $\pm$ 252
All body parts	1543 $\pm$ 11	1320 $\pm$ 109

The results show that randomizing the foot alone leads to a substantial improvement in target performance compared to the no-DR baseline, while largely preserving source performance. In contrast, randomizing proximal components such as the leg or thigh yields smaller gains or even degrades performance. Randomizing all body parts achieves the highest target return overall, indicating that broader exposure to dynamics variations can further improve robustness. Based on these findings, foot-only randomization and full-body randomization are selected for subsequent experiments.

B. Experiment 2: Effect of Randomization Strength

We next investigate how the magnitude of domain randomization affects robustness. Using the best-performing configurations from Experiment 1 (foot-only and all-body randomization), we sweep the randomization coefficient δ .

TABLE II
EFFECT OF RANDOMIZATION STRENGTH δ ON TARGET PERFORMANCE.

Randomized Parts	δ	Source Return	Target Return
Foot	0.05	1323 $\pm$ 138	1004 $\pm$ 39
Foot	0.10	1533 $\pm$ 4	1155 $\pm$ 44
Foot	0.20	1523 $\pm$ 6	1266 $\pm$ 252
Foot	0.40	1678 $\pm$ 15	1105 $\pm$ 71
All	0.05	1627 $\pm$ 10	807 $\pm$ 90
All	0.10	1099 $\pm$ 71	884 $\pm$ 19
All	0.20	1543 $\pm$ 11	1320 $\pm$ 109
All	0.30	1002 $\pm$ 32	695 $\pm$ 2
All	0.40	1208 $\pm$ 22	817 $\pm$ 27

The results reveal a clear trade-off between insufficient and excessive randomization. Small values of δ fail to sufficiently cover the target dynamics, leading to limited improvements

over the baseline. Conversely, very large randomization magnitudes degrade both source and target performance, likely due to increased optimization difficulty and over-regularization. For both configurations, $\delta = 0.2$ yields the best balance between robustness and stability, achieving the highest target performance. This suggests that moderate randomization is critical for effective sim-to-sim transfer.

C. Experiment 3: Effect of Policy Network Architecture

Finally, we analyze the impact of policy network capacity on robustness under domain randomization. Using full-body randomization with $\delta = 0.2$, we evaluate three different MLP architectures across three random seeds.

TABLE III
EFFECT OF POLICY NETWORK ARCHITECTURE ON PERFORMANCE ($\delta = 0.2$, ALL BODY PARTS).

Architecture	Source Return	Target Return
[64, 64]	1551 $\pm$ 81	1048 $\pm$ 238
[128, 128]	1476 $\pm$ 234	1234 $\pm$ 453
[256, 256]	1603 $\pm$ 163	1294 $\pm$ 350

Larger network architectures generally achieve better target performance, suggesting that increased capacity helps the policy represent more robust behaviors under randomized dynamics. However, performance variance across seeds also increases with network size, indicating reduced training stability. The [256, 256] architecture achieves the highest average target return, but at the cost of higher variance, highlighting a trade-off between expressiveness and robustness.

D. Failure and Stability Analysis

We compare three representative policies: a baseline policy trained without domain randomization, a policy trained with moderate domain randomization ($\delta = 0.2$), and a policy trained with excessive randomization ($\delta = 0.4$). Table IV reports the stability metrics evaluated in the target environment.

TABLE IV
STABILITY AND FAILURE ANALYSIS IN THE TARGET ENVIRONMENT.

Model	Timeout (%)	Time-to-Fail	Min z	Angle Std	Near-Falls
No DR	0	366	0.687	0.035	3.3
DR $\delta = 0.2$	100	3000	0.978	0.020	0.0
DR $\delta = 0.4$	0	464	0.686	0.062	3.0

Table IV highlights substantial qualitative differences in policy behavior that are not captured by return-based metrics alone. The baseline policy trained without domain randomization consistently fails due to progressive instability, as indicated by a short time-to-failure and multiple near-fall events. Its minimum torso height remains close to the termination threshold, suggesting that the policy operates with a very small stability margin.

In contrast, the policy trained with moderate domain randomization ($\delta = 0.2$) exhibits fundamentally different behavior. It reaches the maximum episode length without failure in all evaluation runs, maintains a high minimum torso height, and shows a significantly lower torso angle standard deviation. The reduced angle variability indicates smoother and more stable motion, while the absence of near-falls suggests that the policy consistently avoids configurations close to instability.

Finally, the policy trained with excessive randomization ($\delta = 0.4$) shows intermediate behavior. Although its time-to-failure is slightly longer than that of the baseline, it exhibits a much higher torso angle standard deviation. This increased angular variability reflects larger oscillations during locomotion, which ultimately lead to instability and failure. These results indicate that excessive randomization introduces noise into the learned control strategy, degrading stability despite partial improvements in robustness.

While the previous sections focused on return-based performance and stability metrics, these results do not fully explain why certain domain randomization configurations lead to more robust behavior. To gain deeper insight into the internal mechanisms of the learned policies, we now analyze the latent representations produced by the policy networks. This analysis complements the failure and stability metrics by revealing how different training strategies affect the internal encoding of states.

E. Latent Representation Analysis

To explain the observed stability differences, we analyze the internal representations learned by the policy networks. We extract activations from the penultimate layer of the policy network during evaluation rollouts in both source and target environments. These activations are projected to two dimensions using PCA and t-SNE in order to analyze global structure and local neighborhood relationships in the latent space.

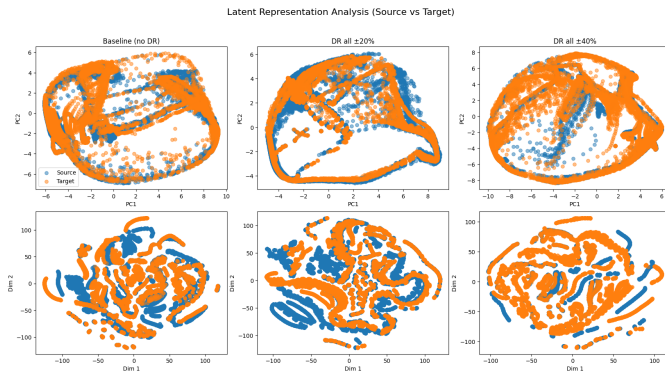


Fig. 1. PCA (top row) and t-SNE (bottom row) projections of latent representations for baseline, moderate DR ($\delta = 0.2$), and excessive DR ($\delta = 0.4$) policies. Source and target states are shown in different colors.

Figure 1 provides a qualitative visualization of the internal representations learned by the policy networks under different

domain randomization settings. Each point in the figure corresponds to the latent representation produced by the policy network for a specific environment state. Importantly, these representations do not correspond to raw observations, but rather to the internal encoding used by the policy to select actions.

The top row shows the projection obtained using Principal Component Analysis (PCA), which preserves the main directions of variance in the latent space and highlights global structural differences. The bottom row shows the projection obtained using t-SNE, which focuses on preserving local neighborhood relationships and is therefore useful for analyzing whether similar states are represented similarly by the policy. In both cases, points are colored according to whether they originate from the source or the target environment.

For the baseline policy trained without domain randomization, PCA reveals a noticeable separation between source and target representations. Although the overall shape of the latent trajectories appears similar, the representations are shifted, indicating that the policy encodes states differently depending on the domain. This suggests that the policy has learned domain-dependent representations, which helps explain its poor stability and early failure in the target environment. The t-SNE projection further supports this observation, showing that source and target states tend to occupy different local neighborhoods, meaning that states which are physically similar are not treated as equivalent by the policy.

In contrast, the policy trained with moderate domain randomization ($\delta = 0.2$) exhibits a markedly different behavior. In the PCA projection, source and target representations largely overlap, indicating that the policy encodes both domains in a similar global structure. The t-SNE visualization further shows that, within local clusters corresponding to meaningful dynamical regimes, source and target states are frequently mixed. This local mixing suggests that the policy has learned domain-invariant features, treating physically similar states from different domains as equivalent. Such representational alignment provides a plausible explanation for the increased stability margin and the absence of near-fall events observed for this policy.

Finally, the policy trained with excessive domain randomization ($\delta = 0.4$) exhibits a different pattern. While source and target representations appear visually mixed, both PCA and t-SNE projections show a more dispersed and less structured latent space. In several regions, source and target clusters exhibit similar shapes but remain spatially separated, indicating that the policy recognizes similar dynamical patterns but fails to align them in a common representation. This loss of structure suggests that excessive randomization introduces noise into the learning process, leading to higher action variability and reduced stability despite partial robustness gains.

Overall, this figure illustrates that effective sim-to-sim transfer is associated not with maximal mixing of representations, but with a balance between invariance and structure. Moderate domain randomization promotes aligned and structured latent representations, which correlate strongly with improved

stability and robust behavior in the target environment.

F. Overall Discussion

Across all experiments, domain randomization significantly improves sim-to-sim transfer performance compared to the no-DR baseline. The results indicate that randomizing distal components such as the foot is particularly effective, likely due to their direct involvement in contact dynamics. Moreover, moderate randomization strengths and sufficiently expressive policy architectures are necessary to fully exploit the benefits of DR. At the same time, excessive randomization or overly large networks can introduce instability, emphasizing the importance of careful hyperparameter selection.

Importantly, the stability and representation analyses reveal that improved sim-to-sim transfer is not merely the result of higher forward velocity or more aggressive behavior. Instead, robust policies operate with a larger margin of safety and encode states in a more domain-invariant manner. These findings suggest that return-based evaluation alone may obscure important qualitative differences in policy behavior.

The latent representation analysis further clarifies the origin of the observed stability differences. Policies trained with moderate domain randomization achieve not only higher target performance, but also exhibit a stronger alignment between source and target representations, which is associated with a larger stability margin. Conversely, excessive randomization leads to less structured latent spaces, providing a plausible explanation for the increased oscillations and reduced robustness observed in this setting.

VI. CONCLUSION

In this project extension, we studied Uniform Domain Randomization for sim-to-sim transfer in a custom Hopper environment. Beyond standard return-based metrics, we introduced a stability-oriented analysis and examined internal policy representations to better understand robustness. Our results show that moderate domain randomization induces domain-invariant latent representations and significantly increases the stability margin of learned policies. Excessive randomization, while increasing variability, degrades representational structure and robustness. Overall, these findings highlight that effective sim-to-sim transfer depends not only on applying domain randomization, but also on carefully controlling its magnitude and interaction with policy capacity.

VII. RELATED WORK

Domain Randomization has been widely adopted as a practical approach to bridge the gap between training and deployment environments in reinforcement learning. By exposing policies to a range of dynamics variations during training, DR encourages the emergence of robust behaviors that generalize beyond a single nominal model.

This idea has been successfully applied in sim-to-real and sim-to-sim transfer settings, particularly for robotic locomotion tasks. Proximal Policy Optimization (PPO) is a commonly used algorithm in this context due to its stability and sample

efficiency. More recent work has explored adaptive and robust variants of domain randomization, aiming to automatically tune parameter distributions or optimize for worst-case performance.

Beyond performance evaluation, several works have emphasized the importance of analyzing internal representations learned by reinforcement learning policies in order to better understand generalization and robustness. Representation analysis techniques such as dimensionality reduction and visualization have been used to study how policies encode states under different training conditions. Although these approaches are not typically used as optimization objectives, they provide valuable insight into the internal mechanisms through which robustness emerges. In this work, we adopt a similar perspective by using PCA and t-SNE as diagnostic tools to analyze the effect of domain randomization on learned representations.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [2] J. Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2017.
- [3] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2018.
- [4] A. Rajeswaran et al., "EPOpt: Learning Robust Neural Network Policies Using Model Ensembles," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [5] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [6] A. Zhang, R. McAllister, and S. Levine, "Learning Invariant Representations for Reinforcement Learning," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.